

Dokumentacja projektu - Baza medycznych badań obrazowych

Desktopowa aplikacja do przechowywania, wyszukiwania i przeglądania

medycznych

Repozytorium: https://github.com/mikita-salauyou/SIM_MedicalDatabase

1. Opis funkcjonalny

Aplikacja realizuje następujące funkcje:

- logowanie do bazy danych (okno logowania, połączenie przez ODBC),
- rejestr pacjentów (dodawanie, edycja, usuwanie, wyszukiwanie po nazwisku),
- ewidencja badań przypisanych do pacjenta (data, typ, opis),
- import pojedynczych plików DICOM oraz całych serii (folderów),
- automatyczne wykrywanie modalności (CT/MR/...) z tagów DICOM,
- przeglądarka DICOM: przewijanie warstw, zoom, regulacja okna/poziomu (W/L),
- eksport listy obrazów do pliku XML (ścieżka, rozmiar, data),
- przechowywanie ścieżek do katalogów ze źródłami danych w bazie.

Pliki DICOM pozostają na dysku w oryginalnych katalogach; w bazie zapisywane

są tylko meta

2. Struktura kodu źródłowego

Cały kod aplikacji znajduje się w jednym pliku main.cpp. Konfiguracja

budowania to

```
SIM/
+-- main.cpp          - cała aplikacja (klasy + funkcje pomocnicze + main)
+-- CMakeLists.txt    - konfiguracja budowania (CMake)
+-- app.ico / app.rc  - ikona pliku wykonywalnego (Windows)
+-- assets/app.png    - źródłowa grafika ikony
+-- data/             - przykładowe dane DICOM (CT-00000, MRI-00000)
+-- start_baza.bat    - uruchamia serwer PostgreSQL + aplikację
+-- release/          - zbudowana aplikacja (exe + biblioteki DLL)
```

Logiczny podział main.cpp:

1. funkcje pomocnicze bazy/DICOM (wolne funkcje),
2. klasy interfejsu (dialogi i widżety),
3. okno główne MainWindow,
4. funkcja main().

3. Struktury danych (schemat bazy)

Schemat jest czteropoziomowy: pacjent -> badanie -> seria -> obraz,

plus pomocn

```
patients (pacjent)
  id PK
  surname, name, birth_date, pesel, sex
  |
  | 1
  |
  | N
studies (badanie)
  id PK
  patient_id FK -> patients.id
  study_date, type, description
  |
  | 1
  |
  | N
series (seria)
  id PK
  study_id FK -> studies.id
  modality, description
  |
  | 1
  |
  | N
images (obraz)
  id PK
  series_id FK -> series.id
  source_id FK -> sources.id
  file_path      (ścieżka WZGLĘDNA do katalogu źródła)

sources (źródło danych)
  id PK
  nazwa
  sciezka  (UNIQUE - bezwzględna ścieżka katalogu na dysku)
```

Pełna ścieżka pliku na dysku odtwarzana jest jako:

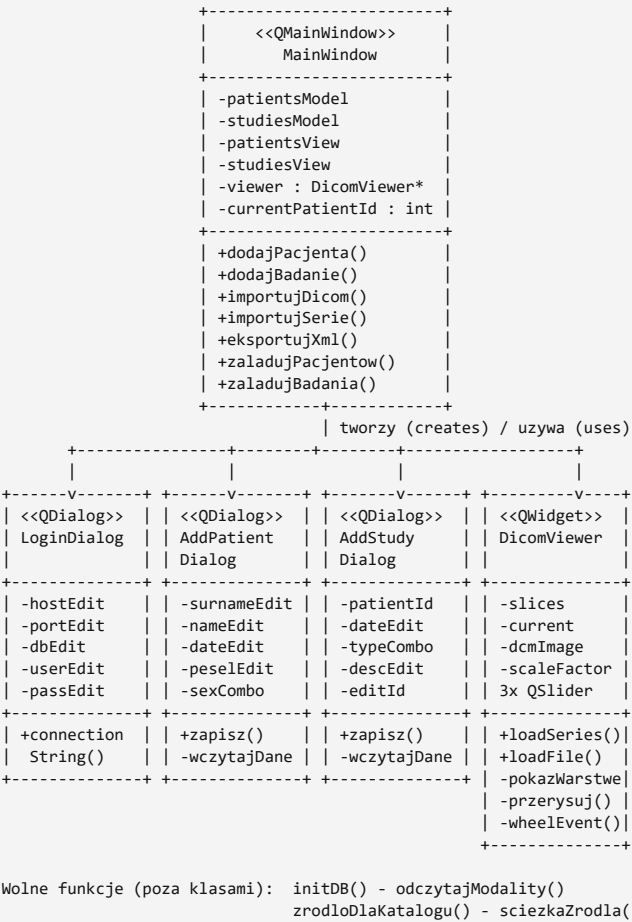
sources.sciezka

Tabele tworzone są automatycznie przy starcie aplikacji (funkcja initDB())

przez CREA

4. Klasy i diagram UML

Aplikacja opiera się na klasach Qt (QDialog, QWidget, QMainWindow).



LoginDialog (QDialog)

Okno logowa

AddPatientDialog (QDialog)

Formularz do

AddStudyDialog (QDialog)

Formularz do

DicomViewer (QWidget)

Przeglądarka

- loadSeries() / loadFile() - wczytanie serii lub pojedynczego pliku,
- pokazWarstwy() - przełączenie warstwy,
- przerysuj() - renderowanie obrazu z aktualnym oknem/poziomem i skalą,
- wheelEvent() - kółko myszy (warstwy) i Ctrl+kółko (zoom).

MainWindow (QMainWindow)

Okno główne

Funkcje pomocnicze (wolne funkcje)

- initDB() - tworzy tabele bazy,
- odczytajModality(path) - odczyt tagu Modality z pliku DICOM (DCMTK),
- zrodloDlaKatalogu(dir) - znajduje lub dodaje źródło dla katalogu,
- sciezkaZrodla(id) - zwraca ścieżkę katalogu źródła po jego id.

5. Funkcje i metody (opis)

Funkcje pomocnicze (wolne funkcje)

Funkcja	Opis
initDB()	Tworzy tabele bazy (CREATE TABLE IF NOT EXISTS); zwraca bool.
odczytajModality(path)	Odczytuje tag Modality (CT/MR) z pliku DICOM.
zrodloDlaKatalogu(dir)	Znajduje źródło dla katalogu lub dodaje nowe; zwraca id.
sciezkaZrodla(id)	Zwraca bezwzględną ścieżkę źródła po jego id.

LoginDialog

Metoda	Opis
LoginDialog(parent)	Buduje okno logowania (serwer, port, baza, user, hasło).
connectionString()	Składa tekst połączenia ODBC (różny dla Windows/Linux).

AddPatientDialog

Metoda	Opis
AddPatientDialog(parent, editId)	Formularz pacjenta; editId>0 = edycja.
zapisz()	Waliduje i zapisuje pacjenta (INSERT/UPDATE).
wczytajDane()	Wczytuje dane pacjenta przy edycji.

AddStudyDialog

Metoda	Opis
AddStudyDialog(patientId, parent, editId)	Formularz badania dla pacjenta.
zapisz()	Zapisuje badanie (INSERT/UPDATE).
wczytajDane()	Wczytuje dane badania przy edycji.

DicomViewer

Metoda	Opis
loadSeries(paths)	Wczytuje serię (listę warstw) i pokazuje pierwszą.

loadFile(path)	Wczytuje pojedynczy plik jako serię z jedną warstwą.
pokażWarstwę(idx)	Przełącza i wczytuje warstwę o indeksie idx.
przerysuj()	Renderuje obraz z aktualnym oknem/poziomem i skalą.
wheelEvent(e)	Kółko = warstwy, Ctrl+kółko = zoom.

MainWindow

Metoda	Opis
onPatientClicked(i)	Klik pacjenta -> ładuje jego badania.
onStudyDoubleClicked(i)	Dwuklik badania -> ładuje obrazy do podglądu.
dodajPacjenta() / dodajBadanie()	Otwierają formularze dodawania.
importujDicom()	Import pojedynczego pliku DICOM do badania.
importujSerie()	Import całego folderu (serii) DICOM.
eksportujXml()	Eksport listy obrazów do pliku XML.
filtrujPacjentow(t)	Filtruje listę pacjentów po nazwisku.
patientMenu(p) / studyMenu(p)	Menu PPM: edycja / usuwanie.
usunPacjenta(id) / usunBadanie(id)	Usuwanie kaskadowe.
załadujPacjentow(f) / załadujBadania(id)	Ładują dane do tabel.

6. Algorytmy

Renderowanie DICOM (okno / poziom). Obraz DICOM ma wartości w dużym zakresie (np.

Nawigacja po warstwach. Seria to lista plików slices. Kółko myszy zmienia indeks

Wykrywanie modalności. Z pierwszego pliku serii odczytywany jest tag DCM_Modaliti

Ścieżki względne i źródła. Przy imporcie katalog pliku porównywany jest z istniejącym

Eksport XML. Zapytanie łączy images z series, studies, patients i sources, od

Usuwanie kaskadowe. Usunięcie pacjenta kasuje najpierw obrazy, potem serie, badania

7. Biblioteki

- Qt 6 (moduły Widgets, Sql) - interfejs graficzny, modele danych, połączenia
- DCMTK 3.6.8 - wczytywanie i dekodowanie plików DICOM (DicomIma
- PostgreSQL 16 - serwer bazy danych. Licencja PostgreSQL.
- psqLODBC (sterownik PostgreSQL ODBC) - warstwa komunikacji ODBC.

8. Kompilacja

System budowania: CMake + Ninja.

Windows (MinGW)

```
$env:Path = "C:\Qt\Tools\mingw1310_64\bin;" + $env:Path
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release `
  -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++ `
  -DCMAKE_PREFIX_PATH="C:\Qt\6.8.1\mingw_64" -DDCMTK_DIR="C:\dcmtdk\cmake"
cmake --build build
```

Po zbudowaniu biblioteki Qt zbierane są obok exe narzędziem windeployqt

(dołącza m.in.

Ubuntu / Linux

Zależności:

```
sudo apt install build-essential cmake ninja-build \
  qt6-base-dev libqt6sql6-odbc libdcmtdk-dev \
  unixodbc odbc-postgresql postgresql
```

Budowanie:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Plik wykonywalny: build/BazaBadanObrazowych.

CMakeLists.txt szuka pakietów Qt6 i DCMTK przez find_package.

Na Linuxie ni

9. Instalacja w systemie

1. Zainstalować PostgreSQL 16 i utworzyć bazę medbaza.
2. Zainstalować sterownik psqLODBC (PostgreSQL Unicode x64) - z pakietu
3. Skopiować katalog release\ z aplikacją na komputer docelowy.
4. Uruchomić serwer bazy i aplikację (skrypt start_baza.bat).

MSI z pos

Tabele tworzą się same przy pierwszym uruchomieniu. Dane importuje się

z poziomu ap

10. Konfiguracja

- Serwer PostgreSQL działa na porcie 5433 (pg_ctl ... -o "-p 5433").
- Dane logowania domyślne: serwer 127.0.0.1, baza medbaza,
- Tekst połączenia ODBC (klasa LoginDialog):
- Źródła danych (katalogi z plikami) dodają się automatycznie przy imporcie.

użytkowni

Windows: